

گزارش سوال ۳ عملی - قسمت ۱ - تمرین ۳ - سیستم عامل

- بخش ۱: فراخوانی سیستمی اولیه

برای این سوال ابتدا از vm استفاده کردیم، اما از آنجا که build کردن هسته سیستم عامل نیاز به فضای دیسک زیادی داشت، نهایتاً از سیستم میزبان که Debian 13 بود استفاده کردیم. هسته ای که از آن استفاده کردیم، همان نسخه linux-6.1.156 بود که در سوال ۳ تمرین ۱ با آن کار کردیم. در فرایند انجام این سوال، directory که در آن هستیم:

```
bagheri-timarchi@debian:~/Desktop/linux-6.1.156/
```

پس از نصب پیش نیازهای لازم:

```
sudo apt update
sudo apt install build-essential kernel-package libncurses-dev bison
flex libssl-dev libelf-dev
```

فایل system call را به صورت زیر ایجاد کردیم:

```
sudo nano kernel/hello_syscall.c
```

تا در سطح هسته پیامی چاپ کند:

```
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(hello_syscall)
{
    printk(KERN_INFO "Hello, world from Bagheri&Timarchi, deep inside
the kernel!\n");
    return 0;
}
```

حال برای اینکه این فایل در هسته کامپایل شود:

```
sudo nano kernel/Makefile
```

و خط زیر را به آن افزودیم:

```
obj-y += hello_syscall.o
```

در فایل زیر، لیست تمامی syscall ها را به همراه نام و id و category آنها مشاهده می کنیم:

```
sudo nano arch/x86/entry/syscalls/syscall_64.tbl
```

سپس این syscall را افزودیم و یک شماره آزاد به آن اختصاص دادیم:

```
451      common  hello_syscall  sys_hello_syscall
```

حال باید این syscall را به header file اضافه کنیم تا کاربر بتواند آن را call کند:

```
sudo nano include/linux/syscalls.h
```

خط زیر را افزودیم:

```
asmlinkage long sys_hello_syscall(void);
```

نهایتاً با دستورات زیر kernel را build کرده و پس از reboot وارد آن شدیم:

```
sudo make menuconfig
sudo make localmodconfig
sudo make -j$(nproc)
sudo make modules_install
sudo make install
```

برای اطمینان از نسخه هسته در حال اجرا:

```
bagheri-timarchi@debian:~$ uname -r
6.1.156
```

در این مرحله یک کد سطح کاربر نوشتیم تا syscall جدید را call کند:

```
nano hello_user.c
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define __NR_hello_syscall 451 // same number as in syscall_64.tbl

int main(void)
{
    syscall(__NR_hello_syscall);
    printf("System call executed. Check dmesg for kernel output.\n");
    return 0;
}
```

```
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$ gcc hello_user.c -o
hello_user
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$ ./hello_user
System call executed. Check dmesg for kernel output.
```

حال با دستور dmesg پیام های سطح هسته را بررسی کردیم:

```
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$ sudo dmesg | tail
[ 971.140741] RBP: ffff888150e47c28 R08: 0000000000000001 R09:
ffffffffff98a47ec7
[ 971.140743] R10: fffffbfff3148fd8 R11: 3d3d3d3d3d3d3d R12:
ffff8881426ee000
[ 971.140746] R13: ffff888141d68000 R14: ffff888104f140d0 R15:
0000000000000000
[ 971.140748] FS: 00007fe22df11680(0000) GS:ffff888450f00000(0000)
kn1GS:0000000000000000
[ 971.140752] CS: 0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[ 971.140754] CR2: 0000000000000000 CR3: 000000013cd8e002 CR4:
0000000000f70ee0
[ 971.140757] PKRU: 55555554
[ 971.140759] note: gst-plugin-scan[10948] exited with irqs disabled
[ 1109.828407] Hello, world from Bagheri&Timarchi, deep inside the
kernel!
```

و مشاهده می کنیم پیامی که می خواستیم به درستی چاپ شده است.

– بخش ۲: توسعه فراخوانی سیستمی

کلیات مراحل این بخش همانند بخش قبل است، بنابراین به تفاوت های موجود اشاره خواهیم کرد. فایل `hello_syscall.c` را به کد زیر تغییر دادیم:

```
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/uaccess.h>

SYSCALL_DEFINE1(hello_syscall, char __user *, buf)
{
    const char msg[] = "This is the second version of the syscall.
    Saying hi directly to the user space!";
    long len = sizeof(msg);

    if (copy_to_user(buf, msg, len))
        return -EFAULT;

    return len;
}
```

که در آن پیامی از سطح هسته به سطح کاربر ارسال می شود. سپس در فایل `header`، خط جدید را جایگزین خط قبلی کردیم:

```
sudo nano include/linux/syscalls.h
```

```
asmlinkage long sys_hello_syscall(char __user *buf);
```

همچنین برنامه کاربر را به کد زیر تغییر دادیم تا با فراخوانی system call جدید، پیام آن را دریافت کرده و در سطح کاربر چاپ کند:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <string.h>

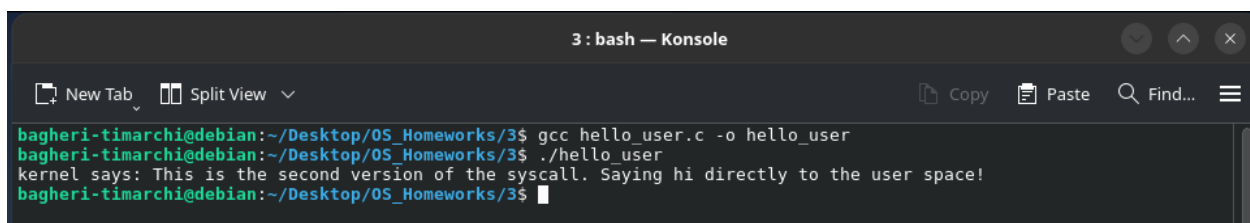
#define __NR_hello_syscall 451

int main(void) {
    char buf[128] = {0};
    long ret = syscall(__NR_hello_syscall, buf);

    if (ret >= 0) {
        printf("kernel says: %s\n", buf);
    } else {
        perror("syscall failed :(\n");
    }

    return 0;
}
```

پس از اجرای کد بالا مشاهده کردیم:



```
3 : bash — Konsole
New Tab Split View
Copy Paste Find...
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$ gcc hello_user.c -o hello_user
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$ ./hello_user
kernel says: This is the second version of the syscall. Saying hi directly to the user space!
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$
```

بعد از اجرای مراحل بالا از ما خواسته شد به جای انتقال پیام از سطح کرنل به سطح کاربر، مستقیماً از همان سطح کرنل پیام را چاپ کنیم. برای این کار صرفاً فایل `syscall` و فایل کاربر را تغییر دادیم. در تصویر زیر هر دو این کدها به همراه نتیجه اجرا قابل مشاهده است:

```
Desktop : bash — Konsole
New Tab Split View Copy Paste Find...
bagheri-timarchi@debian:~$ cd Desktop/
bagheri-timarchi@debian:~/Desktop$ ./hello_world_kernel
This is the second version of the syscall.
Saying hi directly to the user space via stdout!
bagheri-timarchi@debian:~/Desktop$ uname -r
6.1.156
bagheri-timarchi@debian:~/Desktop$ cat linux-6.1.156/kernel/hello_syscall.c
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/fs.h>
#include <linux/file.h>
#include <linux/types.h>
#include <linux/errno.h>

SYSCALL_DEFINE0(hello_syscall)
{
    struct file *file;
    loff_t pos = 0;
    const char msg[] =
        "This is the second version of the syscall.\n"
        "Saying hi directly to the user space via stdout!\n";
    size_t len = sizeof(msg) - 1;
    ssize_t ret;

    // Get file pointer for stdout (fd = 1)
    file = fget(1);
    if (!file)
        return -EBADF;

    // Write to stdout directly (kernel 5.x/6.x safe)
    ret = kernel_write(file, msg, len, &pos);

    // Release the file pointer
    fput(file);

    return ret;
}
bagheri-timarchi@debian:~/Desktop$ cat hello_world_kernel.c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define __NR_hello_syscall 451

int main() {
    syscall(__NR_hello_syscall);
    return 0;
}
bagheri-timarchi@debian:~/Desktop$
```

– بخش ۳: شناسایی محل پیاده‌سازی PCB

PCB ها در کد منبع هسته لینوکس در فایل زیر تعریف شده اند:

```
include/linux/sched.h
```

در واقع PCB یک struct به نام task_struct با تعداد بسیار زیادی attribute است:

```
struct task_struct {  
    volatile long state;  
    void *stack;  
    struct mm_struct *mm;  
    pid_t pid;  
    pid_t tgid;  
    struct task_struct *real_parent;  
    struct task_struct *parent;  
    struct list_head children;  
    struct files_struct *files;  
    struct signal_struct *signal;  
    ...  
};
```

بخش ۴: توسعه ساختار PCB

در این بخش یک فیلد جدید به نام `int new_field;` به ساختار PCB اضافه کرده و سپس با افزودن ۲ `syscall` جدید، مقدار آن را نوشته و یا می‌خوانیم. در فایل زیر، ویژگی مشخص شده را اضافه می‌کنیم:

```
sudo nano include/linux/sched.h
```

```
int new_field;
```

سپس در فایل مربوط به `fork` شدن فرایندها، مقدار این فیلد را برای هر فرایند جدید 0 قرار می‌دهیم:

```
sudo nano kernel/fork.c
```

بعد از خطی از کد که شامل `dup_task_struct()` است، خط زیر را اضافه می‌کنیم:

```
p->new_field = 0;
```

حال `syscall` های مورد نیاز را مانند بخش های قبل ایجاد می‌کنیم:

```
bagheri-timarchi@debian:~/Desktop/linux-6.1.156/kernel$ sudo nano new_syscalls.c
bagheri-timarchi@debian:~/Desktop/linux-6.1.156/kernel$ cat new_syscalls.c
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/sched.h>
#include <linux/uaccess.h>

SYSCALL_DEFINE1(set_new_field, int, value) {
    struct task_struct *task = current;
    task->new_field = value;
    printk(KERN_INFO "set new_field = %d for PID %d\n", task->new_field,
task->pid);
    return 0;
}

SYSCALL_DEFINE0(get_student_field) {
    struct task_struct * task = current;
    printk(KERN_INFO "Get new_field = %d for PID = %d\n", task-
>new_field, task->pid);
    return task->new_field;
}
```



```
-- sudo — Konsole

GNU nano 8.4 kernel/new_syscalls.c
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/sched.h>
#include <linux/uaccess.h>

SYSCALL_DEFINE1(set_new_field, int, value) {
    struct task_struct *task = current;
    task->new_field = value;
    printk(KERN_INFO "set new_field = %d for PID %d\n", task->new_field, task->pid);
    return 0;
}

SYSCALL_DEFINE0(get_student_field) {
    struct task_struct *task = current;
    printk(KERN_INFO "Get new_field = %d for PID = %d\n", task->new_field, task->pid);
    return task->new_field;
}
```

مراحل زیر همانند بخش های قبل هستند:

```
sudo nano kernel/Makefile
```

افزودن این خط:

```
obj-y += new_syscalls.o
```

سپس:

```
sudo nano arch/x86/entry/syscalls/syscall_64.tbl
```

افزودن دو خط جدید:

```
-- sudo — Konsole

GNU nano 8.4 arch/x86/entry/syscalls/syscall_64.tbl
450 common set_mempolicy_home_node sys_set_mempolicy_home_node
# Martin's new syscall for OS-HW3-Q3
451 common hello_syscall sys_hello_syscall
452 common set_new_field sys_set_new_field
453 common get_new_field sys_get_new_field
#
```

تفاوت این بخش با بخش های قبل در این است که نیازی به تعریف `asm linkage` برای این دو `syscall` نیست. در صورت تعریف دستی آن ها، با تعداد زیادی `error` مواجه خواهیم شد.

نهایتاً برای بررسی صحت عملکرد آنچه در این بخش انجام دادیم، کد زیر را ایجاد می‌کنیم:

```
sudo nano new_test.c
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define __NR_set_new_field 452
#define __NR_get_new_field 453

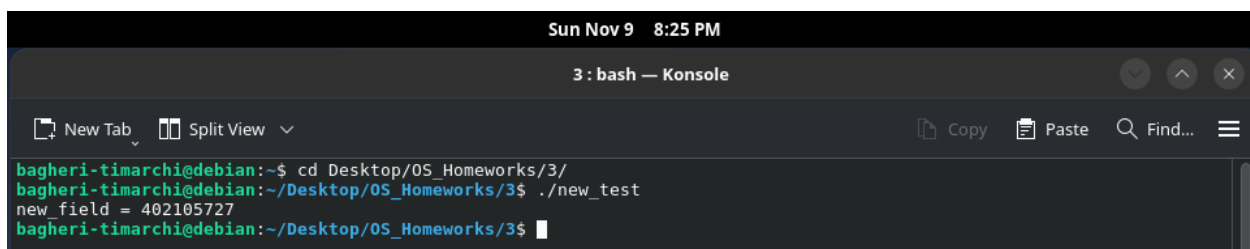
int main(void)
{
    int val = 402105727;
    syscall(__NR_set_new_field, val);

    int ret = syscall(__NR_get_new_field);
    printf("new_field = %d\n", ret);
    return 0;
}
```

```
gcc new_test.c -o new_test
```

```
./new_test
```

و مشاهده شد همه بخش‌ها به درستی کار می‌کنند:

A screenshot of a terminal window titled "3 : bash — Konsole" with a timestamp "Sun Nov 9 8:25 PM". The terminal shows the user navigating to the directory ~/Desktop/OS_Homeworks/3 and running the ./new_test program. The output of the program is "new_field = 402105727".

```
bagheri-timarchi@debian:~$ cd Desktop/OS_Homeworks/3/
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$ ./new_test
new_field = 402105727
bagheri-timarchi@debian:~/Desktop/OS_Homeworks/3$
```